THE KOSELLECK MACHINE

Testing the Sattelzeit as a Network Event

A method note on the Koselleck Machine: measuring whether English vocabulary reorganized as a system, and not only word by word, across 1770-1830.

REPOSITORY	koselleck-networks
DOCUMENT	docs/method.tex → docs/method.pdf
CORPUS SPAN	1500-1900, uniform 20-year windows
DISPLAY RESOLUTION	1.0 (sweep: 7 settings)
COMPILED	August 7, 2026

Contents

1 Context

The historian Reinhart Koselleck argued that the period 1770-1830 (the **Sattelzeit**, German for "saddle period") was not simply an era in which many words happened to change meaning at the same time. He claimed it was a moment when the whole vocabulary of politics and society reorganized **as a system**: concepts shifted together, in relation to one another, rather than drifting independently one at a time. This has always been argued in prose, from close reading of texts. It has never been tested computationally.

Ryan Heuser's *Computing Koselleck* took the first step towards such a test: he trained a separate word-embedding model for each historical period and measured how far each individual word moves from one period to the next. He found that this movement is unusually large around 1770-1830, which supports the idea that something real happened in that window. But his method looks at one word at a time. It cannot say whether words moved **together**, as a system reorganizing around a shared structure, or whether each word's shift is its own independent event that simply happens to cluster in time.

This project adds that second, missing test. Instead of asking how far a single word moves, it asks whether the **groupings** among words, the way the whole vocabulary is organized into clusters of related meaning, change unusually around the same decades. If Koselleck is right, we should see the cluster structure itself reorganize at the Sattelzeit, not just individual words drifting off on their own.

2 Method

The method reuses the same per-period training that Heuser's approach uses, so the two studies remain directly comparable, and adds a network layer on top that a word-by-word approach cannot produce.

1. **Split the corpus into time windows.** The corpus is divided into even 20-year windows, currently 1500 to 1900. The windows are uniform across the whole timeline, not just around the Sattelzeit: if the method measured more finely right where reorganization was expected and more coarsely everywhere else, it would be biased toward finding exactly what it was looking for. Heuser's own results, in fact, show a second spike of word-level change around 1905-1925, unrelated to the Sattelzeit, a pattern that a Sattelzeit-only sampling scheme could never have picked up. The 1500-1900 span reflects the corpus assembled for this project so far, not a limit of the method itself: the same pipeline applies unchanged to any other span or window

size, set entirely in configuration, given a period-dated corpus to feed it.

2. **Train a language model per period.** For each time window, a model (**word2vec**) is trained independently, learning a numeric representation of every word from how it is actually used in that period's text: words that appear in similar contexts end up with similar representations. Training a separate model per period, rather than one model updated continuously, is what makes it possible to later measure both individual word drift and whole-system reorganization: a word's meaning only makes sense relative to the other words trained alongside it, in that same period.

3. **Build a word-similarity network.** From those representations, a network is built for each period: every word is a node, linked to its 15 closest neighbours by **cosine similarity** (a standard measure of how alike two words' representations are). A fixed number of nearest neighbours is used instead of a similarity threshold, because at this vocabulary scale a fixed threshold produces networks that are almost fully connected: tens of millions of edges per period, unusable and with no meaningful structure to cluster. Purely grammatical words (articles, prepositions, pronouns) and single-letter tokens are excluded as network nodes, since the interest is in words that carry conceptual content, not the grammatical scaffolding of the language (those grammatical words are still used during training itself, since they help place every other word correctly).

4. **Detect communities.** A **community detection** algorithm (**Leiden**) is run on each network, grouping words that are densely connected to one another. In practice, these groups function as semantic fields: sets of words that, in that period, are used in similar ways to each other more than to the rest of the vocabulary.

   This differs from familiar clustering methods such as k-means in a basic way. k-means fixes a number of groups in advance and assigns each item to whichever group's average position it sits closest to. Leiden does neither: it does not fix the number of groups beforehand, and it never compares a word to a group's "average", it only ever asks a relational question about a word's actual connections. For any word, and any group it is already directly connected to, Leiden checks whether moving that word into the group would make the network as a whole denser *within* groups and sparser *between* them than plain chance would predict, given how connected that word and that group already are overall. If the move helps, it is made; if not, the word stays put or a different move is tried. Repeating this, word by word, until no single move can improve things any further, is the entire algorithm.

   The number this process is trying to improve at every step has a name, **modularity** (written Q). It compares how connected the words placed inside the same group actually are against how connected they would be expected to be if the same set of

connections had been handed out at random instead. A grouping that captures real structure scores well above zero; a grouping no better than chance scores around zero; a grouping that forces genuinely unrelated words together scores *below* zero, actively worse than not grouping at all. Leiden's search is nothing more than hill-climbing on this one number.

5. **Repeat detection at several levels of detail.** Community detection does not have one single "correct" answer: a parameter called **resolution** controls how many groups appear and how large they are, by changing how large a word's own connections have to be, relative to chance, before joining a group is judged worthwhile. For that reason every network is processed at seven different resolution settings, and any claim about reorganization has to hold up across most of those settings, not just one chosen after the fact.

   > **Hard rule.** No single setting among the seven is ever treated as "the" result and the rest discarded. The sweep's entire purpose is to check that the same conclusion holds broadly across settings, not to pick whichever one happens to agree with the hypothesis. No finding is reported if it only holds at a single, convenient resolution.

   One setting, resolution 1.0 (plain textbook modularity, with no bias toward larger or smaller groups) is used separately, only to choose a single picture to display in the interactive explorer described below. That is a display choice with no bearing on which finding gets reported as real, and it is not an arbitrary one: looking across the already-computed sweep, modularity (how well a grouping captures real structure, against random chance) sits on a broad plateau from resolution 0.5 to 1.5 and peaks, only marginally, at 1.0, while the migration fraction rises and period-to-period stability falls fairly steadily as resolution increases beyond a low setting. A higher display resolution would therefore mechanically inflate the very reorganization measurement this project is testing, which is reason enough to keep the display fixed at the standard default rather than a value chosen after seeing the result.

6. **Compare consecutive periods.** For each pair of consecutive periods, the comparison is restricted to words that appear in both (each period has its own vocabulary). The best possible match between one period's groups and the next period's groups is found first, because group numbers do not mean the same thing from one period to the next, so the correspondence has to be established before anything can be compared. Concretely (`metrics.py`'s `align_communities`): build a contingency table counting, for every (old group, new group) pair, how many shared words fall in that pair, then solve it as a single global assignment problem (the Hungarian algorithm,

via SciPy's `linear_sum_assignment`) that picks the one-to-one relabeling of new group numbers onto old ones maximizing the total word count matched. This is a fixed, deterministic optimum over all groups at once, computed once per period pair – there is no percentage-overlap threshold anywhere in it, and no per-group decision made independently of the others. A single shared word then counts as **moved** if, under that one relabeling, its new group's mapped-back old identity differs from the old group it was actually in; otherwise it counts as **stayed**. Then the fraction of shared words that still ended up in a different group, despite that best match, is measured. That fraction, how much of the shared vocabulary "migrated" to a new group, is what this project calls the **migration fraction**: its central measurement of how much reorganization happened between two periods. The interactive explorer's per-word "moved"/"stayed" arrows (Section 3) are read off this exact same per-word classification, not a separate or looser check, so a single word's arrow and the aggregate migration fraction can never disagree about what counts as moving.

7. **Test whether reorganization concentrates at the Sattelzeit.** The hypothesis predicts that the migration fraction should be unusually high specifically for the transitions that fall inside or close out the 1770-1830 window, compared to every other transition across the full 1500-1900 timeline.

8. **Repeat the whole measurement per region.** The corpus is not evenly British across the timeline, and the imbalance is worst exactly where it matters most: in 1780-1800 the American material (Evans-TCP, 1681 documents) outweighs the British (ECCO-TCP, 910 documents), because ECCO's public transcriptions thin out sharply at the end of the eighteenth century. A change in the cluster structure across that boundary could therefore reflect a change in *who was writing* rather than a change in the language. To separate the two, every step above (training, network construction, community detection, the resolution sweep, and the migration fraction) is also run on each region's documents alone, in addition to the combined corpus. If the reorganization is real, it should show up inside the British-only and the American-only timelines as well, not only in the mixture of them; if it shows up only in the combined run, it is an artifact of corpus composition rather than semantic history. Nothing here hardcodes which regions exist: a document's region is simply the top-level folder its source was filed under, so the same check extends to any other split of the corpus.

9. **Dictionary cross-check (not yet implemented).** As a second line of evidence, independent of the network entirely, the plan is to take the words that change group right at the pivot and check whether the OED or other period dictionaries independently record a dated sense change at the same moment. This layer is not yet designed: the corpus mixes British and American sources, and matching each

source against the wrong regional dictionary would invalidate the comparison, so the choice of dictionary per source has to be resolved first.

2.1 Corpus

The primary source is the **Text Creation Partnership (TCP)**: curated, manually corrected transcriptions with no OCR noise, covering 1500-1800 (EEBO-TCP, ECCO-TCP, Evans-TCP). Since the Sattelzeit extends to 1830 and TCP stops at 1800, the 1800-1900 gap is filled by the **British Library**'s digitised 19th-century books collection (49,455 books, roughly 65,000 volumes, OCR-derived text with catalogue metadata already attached, public domain). Project Gutenberg was considered first and dropped: its metadata carries only its own digitisation date, not the original print publication date, which makes it unusable for a method that buckets every document by the year it was actually written. HistWords (pre-trained historical embeddings) is used only as an independent sanity check, not as primary evidence.

Each source is filed under a region: `british` for EEBO-TCP, ECCO-TCP, and now the British Library supplement; `american` for Evans-TCP, which stops at 1800 along with the rest of TCP. The region tag is not a claim about two different English-language communities - for most of this timeline, American print culture and British print culture share the same language, and treating them as linguistically distinct would not be defensible. It tracks archive provenance instead: EEBO/ECCO and the British Library are one continuous British-print lineage, Evans is an independently curated American-print archive. Read this way, the region split is a robustness check, not a dialect comparison: if the reorganization signal this project measures shows up independently in both archives, that is evidence it is not an artifact of one archive's quirks; the combined corpus stays the headline result either way. A direct consequence of this design: since the British Library supplement is tagged `british`, the `american` region has no data past 1800 at all, an honest asymmetry rather than a gap to paper over.

2.2 Not a closed dataset

This project is a method with one corpus attached, not a finished corpus that happens to have a method. The repository ships the pipeline; the corpus is fetched separately and is deliberately not vendored. That has a practical consequence worth stating plainly: a reader who disagrees with the corpus, or who simply has a better one, can rerun the whole measurement on their own text rather than take these numbers on trust. There are exactly two cases, and only one of them touches code.

Case A: the material is already in TCP's P4 XML format (another TCP release, or a repackaged TCP subset). No code change at all. The zip files are dropped at

`<data_root>/corpus/tcp/regions/<region>/<source>/*.zip` and the pipeline is rerun from `src/parse_tcp.py`, which discovers regions and sources by scanning that folder tree rather than from any registry. Both folder names are free-form: the region level becomes the region tag carried through the entire pipeline and the webapp's region toggle, and the source level is a label kept in the manifest for diagnostics. Shards whose filename marks them as TCP "unedited" variants are skipped automatically.

Case B: the material is in any other raw format (plain text, JSON, a different XML flavour, OCR dumps). One file changes, `src/parse_tcp.py`. A reader function is added next to the existing `iter_xml_members` that walks the new format and, for each document, emits the same normalised record the TCP path already writes to `processed/all_docs.jsonl`:

```
{"region": "...", "source": "...", "doc_id": "...", "year": 1782, "text": "..."}
```

plus the matching `region,source,doc_id,year,chars` row in `manifest.csv`. Documents with no extractable year or no text are dropped, exactly as in the TCP path. That is the entire contract: everything downstream, `bucket_periods.py`, `embeddings.py`, `network.py`, `community.py`, `metrics.py`, and the webapp, reads only those normalised records and needs no change. Period boundaries and model hyperparameters are configuration, not code, and live in `config.yml`. In both cases, if the new material introduces a region name that did not exist before, the region-split outputs and the webapp's region toggle pick it up on their own; there is no list of valid regions anywhere to update.

To be clear about what this is not: there is no upload interface, no hosted service, and no API-key-driven ingestion. The mechanism is the ordinary one, clone the repository, put corpus files in the right folder, rerun the pipeline scripts in order.

3 The interactive explorer

Alongside the pipeline, a small web tool, the **Koselleck Machine**, makes the per-period networks directly explorable, without reading raw numbers. It has three views of the same underlying data.

Timeline. The primary view. Type a word and see, in one continuous strip, which community it belonged to in every period it appears in: the community's plain-English label and its lane (below) as a secondary colour cue, an explicit gap where the word is simply absent from that period's vocabulary, and a marked step wherever the word's community actually changed from the period before, versus stayed the same. Earlier drafts of this tool led with the graph explorer below, but historian collaborators who reviewed it found a force-directed diagram hard to read at a glance; what they consis-

tently asked for instead was exactly this, which domain a word belonged to, over time, read directly rather than inferred from a picture. Clicking any period in the strip opens that word's full neighbourhood there, reusing the word search view below.

Word search. The same underlying data, presented as a plain table instead of a diagram: type a word and a period, and get back its closest neighbours by similarity, each neighbour's community, and whether the searched word's own community changed since the previous period. Useful when the actual numbers matter more than the picture, and reused directly by the timeline's own per-period drill-in above.

Graph explorer. A period slider drives a force-directed network diagram: nodes are words, their size reflects how connected they are, and their color marks which community they belong to in that period. A search box focuses the view on one word and its neighbourhood; moving the slider one period at a time shows that neighbourhood evolving, and a change in a word's color between periods marks a real change of community, not just a coincidence of the diagram's layout. A side panel explains the method in plain terms and shows, for the currently selected resolution, how the migration measurement holds up across all seven resolution settings, the same robustness check described above, made visible rather than just claimed. Kept for browsing the raw network structure, and because building it first is what surfaced this whole labeling and lane apparatus, but no longer the primary way to read the tool, per the reception above.

Region toggle. Where the per-region runs described above have been built, both views also offer a region switch (combined, British only, American only) so the same word in the same period can be compared across the sub-corpora directly, and a shift can be checked against the possibility that it only exists in the mixture. The switch lists only the regions that were actually built, read off the data itself rather than assumed.

Every community shown in any view carries a short, plain-English label (for example "Government & Law", "Moral Character & Virtue"). These labels were generated by having a language model read each community's most representative words and summarize the shared theme in a few words: a convenience for a human reader, not a finding in themselves. A label is not re-read independently every period, however. Whenever a community's best correspondence to the period before it (the same correspondence step the migration fraction is built on) can be established, its label is inherited unchanged from that predecessor, and a fresh read only happens for a community with no such predecessor: the first period a region's data begins, or the side of a genuine reorganization that a word moved into. In the combined corpus, roughly nine in ten community labels are inherited this way rather than independently generated for a given pipeline run; the per-region runs behave similarly. This keeps a community's displayed name consistent with whether the tool has actually flagged it as having

moved or stayed, rather than the two being computed separately and occasionally disagreeing, which an early version of the tool did. Roughly half of communities are sorted, by the same process, into the **“Structural / Uncertain”** lane rather than a topical one (down from 70% before the labeling resolution was raised, see below): the corpus is multilingual (English, Latin, Law French, Welsh, Scots, and more) and early modern, and a community detected by this method sometimes groups words by shared grammar (verb conjugations, comparative forms, spelling variants) rather than by shared topic. Every view surfaces this lane directly next to the labels, rather than presenting every label as equally reliable.

Each community is additionally sorted into one **lane**: a small, fixed list of fifteen broad domains (government and law, religion, medicine, trade, and so on), plus one explicit “structural / uncertain” lane for exactly the “mixed” case above. The lane list itself was authored once by hand, from a read-through of every non-mixed label already produced, then fixed. A language model assigns each community to one of these existing lanes; it never invents a new one per run. That fixed-in-advance property is what lets a lane stay a meaningful, comparable signal across independent pipeline runs even though, as the last section explains, the underlying community structure and the raw label text do not reproduce identically run to run. In the tools above, a community’s plain-English label and number stay the primary signal; its lane is a secondary colour cue layered on top, not a replacement for reading the actual label.

3.1 How the labels are actually produced

The paragraphs above describe what the labels are for. This subsection gives the mechanics, so that a technical reader can see exactly what the model was asked to do and reproduce or replace it. Labeling is an optional, clearly separable stage: every quantitative result in this document is produced by `src/metrics.py` and needs no labels at all.

3.1.1 Choosing the labeling resolution

Community detection produces a partition at every resolution in the sweep; the webapp displays and labels only one of them, a choice separate from the sweep itself and revisited once already. It was originally 1.0. Adding the British Library supplement grew some periods’ vocabularies past 170,000 words, and at 1.0 the largest community in the latest periods had grown with them to 24,608 words (14% of that period’s vocabulary) - a size at which no label, model or human, can describe what is actually inside it. This was not a new failure mode introduced by the supplement, only a more visible one: the largest community was already a stable 12-15% of the vocabulary at every period size tested, small and unremarkable at 70,000 words, absurd at 170,000.

Modularity falls off smoothly as resolution rises, with no sharp elbow in the tested range, so there is no purely statistical answer for the right resolution - the real ceiling is practical, how many communities per period stay browsable, not mathematical. The choice was settled by a concrete before/after check rather than a formula. Take the community containing the word “system” in 1870-1890, the single worst offender: at resolution 1.0 its top words read as an unstructured mix (*perjury, arrogance, indigestion, sensuality, bronchitis, extortions, dyspepsia*); at 2.0 and 3.0 the same words are still there, effectively unchanged; only at **4.0** does the community actually split, and the half that keeps “system” becomes a real, nameable theme (*arrable, assessments, registration, payment, loans, forfeitures, coroners, judiciary, treasurers, sheriffs, freehold, manors, copyhold*) - civil administration and property law. The same check on two other periods at 4.0 (one small, one mid-sized) produced equally coherent results (philosophy/rhetoric/mathematics; church governance and clergy), so the resolution was raised project-wide rather than only for the largest periods. `config.yml`’s `leiden.label_resolution` now reads 4.0; `leiden.resolution_sweep`, the range every quantitative claim is actually tested across, was extended to match (up to 4.0, from 2.0), so the display choice and the tested range stay aligned. Growing the corpus further should be expected to call for raising this again - the right resolution scales with vocabulary size, it is not a constant.

3.1.2 The prompt

The prompt lives in the repository as a single Markdown file, `src/prompts/community_labeling_v1.md`, which is parsed at runtime by `src/label_communities.py` rather than duplicated as a string constant in code, so the file stays the one place the instructions and the lane list are edited. It has three sections: the fixed lane list, the system prompt, and the user message template. One model call is made per Leiden community, at the resolution set in `config.yml` (4.0, see above).

The system prompt, quoted from that file, tells the model:

> “You are labeling word clusters from a computational study of an early modern and eighteenth/nineteenth-century English-language corpus (the Text Creation Partnership: EEBO, ECCO, Evans). Each cluster is a ‘community’ found by running the Leiden algorithm on a per-period word-similarity network, words that are used in similar contexts more than they are used with the rest of that period’s vocabulary. The corpus is multilingual (English, Latin, Law French, Welsh, Scots, and other languages appear untranslated) and pre-modern, so a community sometimes groups words by shared grammatical form (verb conjugations, comparatives, spelling variants), by shared foreign language, or by proper names (people, places, biblical figures) rather than by a real topic. *That is an honest and expected outcome, not a failure to avoid.*”

It then asks for exactly three decisions:

1. “Whether most of the top words share a genuine topical or thematic connection a historian would recognize (e.g. law, medicine, religion, trade), as opposed to being grouped mainly by grammar, a shared non-English language, or being mostly proper names or fragments.”
2. “A short plain-English label, two to five words, for what the community is actually about. If the grouping is grammatical, foreign-language, or name-driven rather than topical, the label should say what kind of cluster it is (e.g. ‘Second-Person Verb Forms’, ‘Latin Text’, ‘Biblical Names’), not force a topic onto it.”
3. “Exactly one lane from the fixed list above. Use ‘Structural / Uncertain’ whenever the answer to (1) is no, do not stretch a grammatical, foreign-language, or name cluster into a thematic lane just because a few of its words are suggestive of one.”

The user message carries only the material the model is meant to judge, filled in from the extraction step: the region, the period, the community size in words, and the community’s top 25 highest-degree words, most connected first.

Two properties of this setup matter more than the wording. First, the lane vocabulary is closed. The model is not asked to invent a taxonomy; it must choose one of fifteen lanes fixed in advance, listed in the same file and reproduced here in the order the prompt gives them:

1. Government, Law & Administration
2. Religion, Theology & the Church
3. Morality: Virtue & Vice
4. Medicine, Body & Health
5. Science, Mathematics & Natural Philosophy
6. Nature, Landscape & Weather
7. Military & Warfare
8. Trade, Finance & Commerce
9. History, Genealogy, Nobility & Kinship
10. Rhetoric & Persuasion
11. Literature, Drama & Poetic Diction
12. Domestic Life, Dress & Household
13. Geography & Territory
14. Books, Learning & Scholarship
15. Structural / Uncertain

That list was not invented for the prompt either: it was derived by hand from 707 labels already produced across the combined, British, and American runs, then frozen. Second, the response is not free text. The model must answer by calling a tool, `assign_label`, whose schema takes a `label` string ("two to five word plain-English label") and a `lane` constrained by a JSON-schema `enum` to exactly the fifteen lanes parsed out of the prompt file. A returned lane outside that list is rejected and the call retried, so a label with an out-of-taxonomy lane can never reach the data files.

3.1.3 The workflow

Labeling runs as two scripts and three subcommands, after `src/community.py` has produced the community assignments:

```
python src/extract_community_words.py
python src/label_communities.py generate --region combined
python src/label_communities.py compile --region combined
python src/label_communities.py publish --region combined
```

`extract_community_words.py` writes, for each populated period at the configured label resolution, every community's size and its top 25 highest-degree member words to `communities/community_words_res4.0[_<region>].json`. It runs once for the combined corpus and once per region discovered on disk, because a region's own Leiden run has its own community numbering and therefore needs its own words file and its own labels file.

`generate` turns that JSON into a human-editable CSV with one row per community and the columns `region`, `period`, `community_id`, `n_words`, `top_words_preview`, `label`, `lane`, `origin`, `inherited_from`. It walks periods in chronological order and, for each community, first asks whether the Hungarian alignment that `metrics.py` already uses for the migration fraction maps it back to a predecessor community that already has a label. If so, and the inheritance chain is still short enough (see "Bounded reclassification" below), the label and lane are inherited verbatim (`origin=inherited`: free, deterministic, no model call). A community needs a fresh read for one of two reasons: it is a genuine genesis (a region's first labeled period, or the moved-into side of a real reorganization, `inherited_from` left empty), or it is a continuing lineage whose inheritance chain just hit the cap (`inherited_from` stays populated). Either way the row is either left blank for a human or an agent to fill in by hand (`--fill blank`, the default, and this project's actual practice) or filled by one API call each (`--fill llm`). Rows whose `origin` is `human` are never overwritten; `--overwrite` re-fills only the blank and model-generated rows. In the normal flow `generate` is run twice: once to see which rows come back blank, then again after they are filled, which resolves every inheriting row downstream of them for

free.

Bounded reclassification. Verbatim inheritance solved the problem it was built for (Section 3.1, “Label continuity across periods”), but introduced a different one: nothing ever forced a label to be checked against current content again, so a chain could inherit unbroken for as long as the underlying community kept structurally matching a predecessor. Found in practice, 2026-08-07, while investigating a collaborator’s report that a community’s name “made no sense”: a community genuinely Dutch/German text at 1510-1530 (*godt, niet, christus, gott*) was still labeled “Dutch and German Text” fifteen periods later, at 1810-1830, by which point its top words were about church governance (*romish, deacons, clergymen, doctrines*). Walking the chain back turned up seven further, completely unrelated themes in between - Scots legal prose, an Anglo-Scottish border-warfare narrative, romantic verse, classical mythology, early Christian figures, early Popes, and emotional-distress vocabulary - none of it ever re-read. The structural signal was never wrong: that community’s best-matching predecessor really was the same community, every single step. Only the label, carried forward unexamined, was.

`generate` now caps how many consecutive periods a label may be inherited before forcing a fresh read (`label_communities.MAX_INHERITANCE_CHAIN`, currently 3, chosen by judgement rather than measured: short enough that a fifteen-period chain could not recur unnoticed, long enough that a genuinely slow-changing lineage is not re-read for no reason). When the cap is hit, `inherited_from` stays populated - the row is still visibly a continuation of the same tracked lineage, not a new entity - but the label and lane are re-derived from that period’s own current top words, exactly as a genesis community would be. This does not touch `align_communities` or the migration fraction at all: “stayed” still means exactly what it always meant, structurally, and the resolution sweep behind every quantitative claim is completely unaffected. Only how long a *label* may go without being re-examined is bounded now, the same way a museum periodically re-examines, and occasionally reclassifies, a specimen without disputing its provenance - the metaphor this project’s own visual design already uses. The interactive explorer’s own “stayed” arrow gained a third, narrower case for exactly this: still “stayed” (the structural claim is unchanged), but its tooltip now says the description was just re-examined against current words rather than carried forward unread, distinct from the ordinary case where nothing changed at all.

`compile` turns the CSV, including any hand edits, into `communities/community_labels_res4.0[_<region>].json`, which is the exact file `webapp/app.py` reads. It also stamps a `_meta` block into that JSON recording the resolution, the region, the number of communities, how many were human-edited, how many were inherited, and a standing caveat that the labels are a single model read-through and not an empirically vali-

dated taxonomy. Compiling used to append a literal "(mixed)" to the label text of every `Structural / Uncertain` community; that is no longer done. The `lane` field already carries the "this is not a real topic" signal on its own, and stacking the word "mixed" onto an already-decisive reading ("Second-Person Verb Forms", "Biblical Book Abbreviations") made even a clear description look like the model could not settle on one.

`publish` copies the CSV and the compiled JSON for a region out of the data directory and into this repository. `--region` also accepts a region name or `all` at every subcommand.

3.1.4 Where the published labels live

The label files that the pipeline writes live in the data directory, which is gitignored and does not travel with the repository. That is why a collaborator cloning the repo previously saw no community names at all. The `publish` subcommand exists to fix exactly that: it copies a small (roughly 60 KB total) snapshot into the repository's `labels/` directory, which currently holds one CSV and one compiled JSON per region:

```
labels/community_labels_res1.0.csv
labels/community_labels_res1.0.json
labels/community_labels_res1.0_british.csv
labels/community_labels_res1.0_british.json
labels/community_labels_res1.0_american.csv
labels/community_labels_res1.0_american.json
```

That is the citable snapshot: the version of the labels this document describes, readable without running the pipeline or holding the corpus. `publish` only copies files to disk; it never commits, so the snapshot is reviewed by hand before it is added to version control.

4 Current limitations

Five limitations shape what can be claimed right now, and all five are visible in the tool itself rather than hidden.

Corpus coverage. 1800-1900 is now covered by the British Library supplement described above, but not evenly: ECCO is thin after 1780, so the 1780-1810 window still leans on more American than British documents even with the supplement in place, and the `american` region has no data at all past 1800, since Evans-TCP stops there and the supplement is British-only (see Corpus above). Per-region results therefore carry wider uncertainty than the combined headline numbers,

and the `american` region specifically cannot speak to anything past the Sattelzeit's closing edge.

OCR artifacts. A separate, still-open issue sits inside the British Library text itself, not in its date range: that text is OCR-derived, unlike TCP's manually corrected transcription, and OCR introduces its own artifacts. One class, an original line-break hyphen left sitting in the text (*"utrum- que"* for *utrumque*), was found and fixed by rejoining a letter-hyphen-whitespace-letter pattern before tokenizing. A second, distinct class was not: the OCR sometimes drops the line-break hyphen entirely and leaves a bare space instead (*"par ticulars"* for *particulars*), which carries no punctuation signal left to detect it by. This produces communities that are nothing but word fragments rather than a real topic - confirmed directly in the 1810-1830 period, at least fifteen such communities, some several hundred words large. The labeling pass below handles this honestly (routed to the "Structural / Uncertain" lane rather than mislabeled), but the underlying vocabulary and network metrics for 1810-1830, 1830-1850, and 1870-1890 are still diluted by it. A proper fix needs something more than pattern matching on punctuation - most plausibly, checking whether an adjacent pair of short, otherwise-unusual tokens concatenates into a real English word - and has not been built yet.

A separate, unconfirmed TCP transcription artifact. Raising the labeling resolution (below) surfaced a third, distinct word-splitting pattern in a handful of communities in TCP-covered periods (1510-1650), which is *not* the British Library issue above - TCP is manually keyed, not scanned. The pattern is specific: a "con-"/"com-" prefix is missing from otherwise-recognizable words (*mytted* for *committed*, *uersation* for *conversation*, *monwealth* for *commonwealth*). The most likely cause is TCP's own markup for a scribal abbreviation or suspension mark, common for exactly this prefix in early modern print, being mishandled during text extraction - but this has not been confirmed against the raw XML, and no fix has been attempted. Flagged here rather than fixed blindly.

Label reliability. A community's label is inherited, not re-read, for as long as its best correspondence to the previous period holds. Only a minority of labels (roughly one in ten to one in six, depending on region) are an independent read for a given pipeline run. That independent read is still a single, unvalidated pass by a language model, not a checked ground truth: useful for orientation, not

for citation. Inheritance carries its own, different cost. On the corpus's longest-running community lineages, a label can go up to fifteen periods (close to three hundred years) without being independently re-read, and the community's actual word composition can drift well past what the label still says by then, a staleness problem rather than a wrong-read one, and not yet addressed.

Reproducibility. Rerunning the full pipeline from scratch on the same corpus does not reproduce an identical result, and this is worth stating plainly rather than glossing over. Community detection itself is properly seeded and deterministic given a fixed network; the gap sits one step earlier, in training. Language-model training here uses multiple parallel threads for speed, and that kind of parallel training is not bit-for-bit repeatable even with a fixed random seed, because the exact order in which threads update shared values depends on how the operating system happens to schedule them that run. Small differences in the trained word vectors cascade into a similarity network with slightly different edges, which cascades into a structurally similar but not identical community partition. In practice this shows up as, for instance, one full rerun finding 304 communities in the combined corpus where an earlier run found 308: a real difference, not a bug, and not one worth trading away the threefold training speed of parallel execution to chase, since it does not bear on the actual claim under test. That claim is whether reorganization concentrates at the Sattelzeit, tested by the resolution sweep described above, holding across settings within one run, and not whether a specific community keeps the same id, or even the same member words down to the last one, across two independently retrained copies of the pipeline. The one thing designed to stay meaningfully comparable across independent reruns is the small, fixed lane list described above, precisely because it is authored once by hand and never reinvented per run.